

Spine Large-Graph Campaign Results

Correctness-Gated Spine vs. GraSU+ReGraph K1/K4-shared

Chuxiao Han

Live campaign snapshot, July 2026

Abstract

This report is generated directly from correctness-gated campaign artifacts. The snapshot is **PARTIAL**: 46 of 95 planned physical executions are currently admitted, with 4 complete three-system groups. Missing bars are unexecuted comparisons, not zero-valued measurements. Every plotted row passes exact integer-state comparison or the declared 10^{-5} float32 PageRank output tolerance. Headline plots use insertion batch 8; the update sweep labels operation and batch size explicitly.

1 End-to-end performance

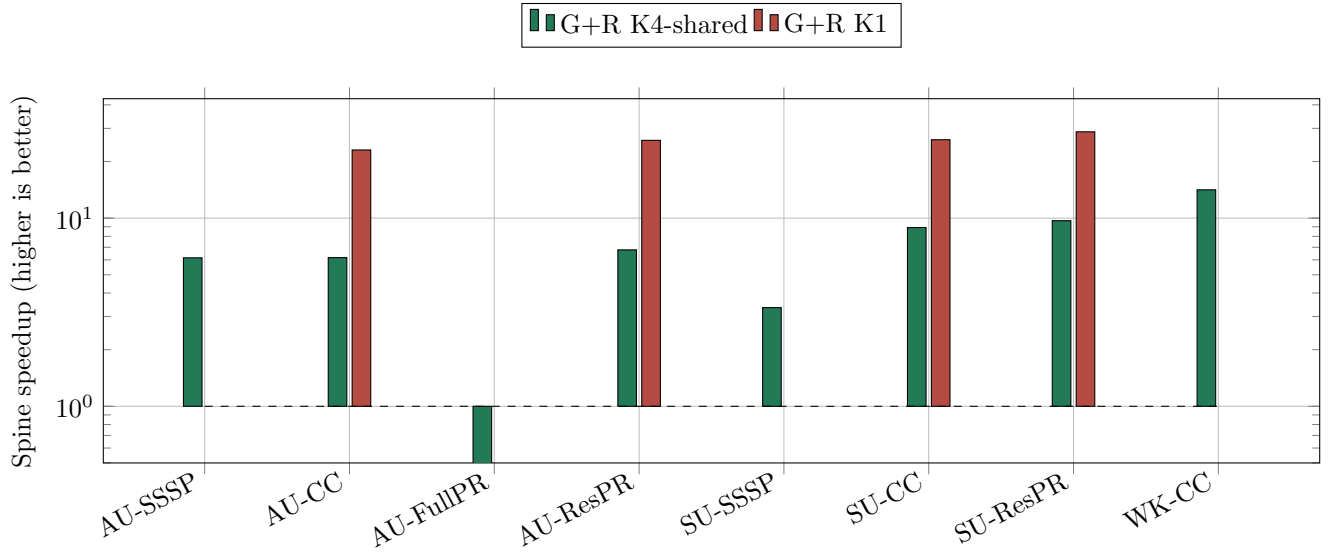


Figure 1: Device-cycle speedup after both architecture and independent mathematical correctness gates. K4-shared is the primary GraSU baseline.

2 Memory traffic and HBM energy

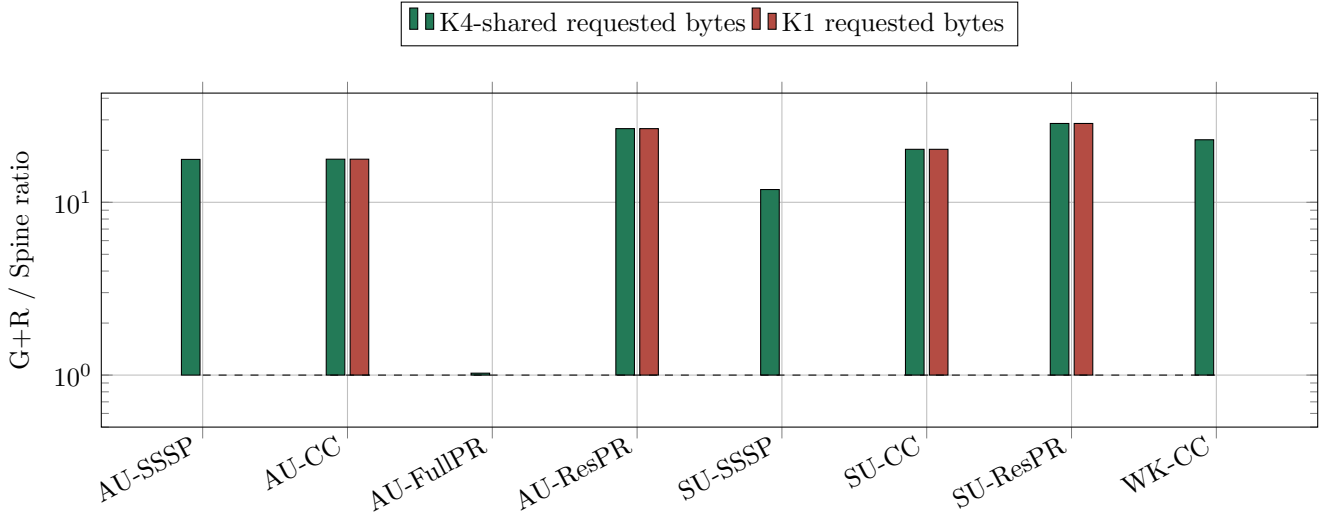


Figure 2: Accepted backend bytes. Ratios above one mean GraSU+ReGraph moves more data than Spine; request and DRAM completion ledgers close on every row.

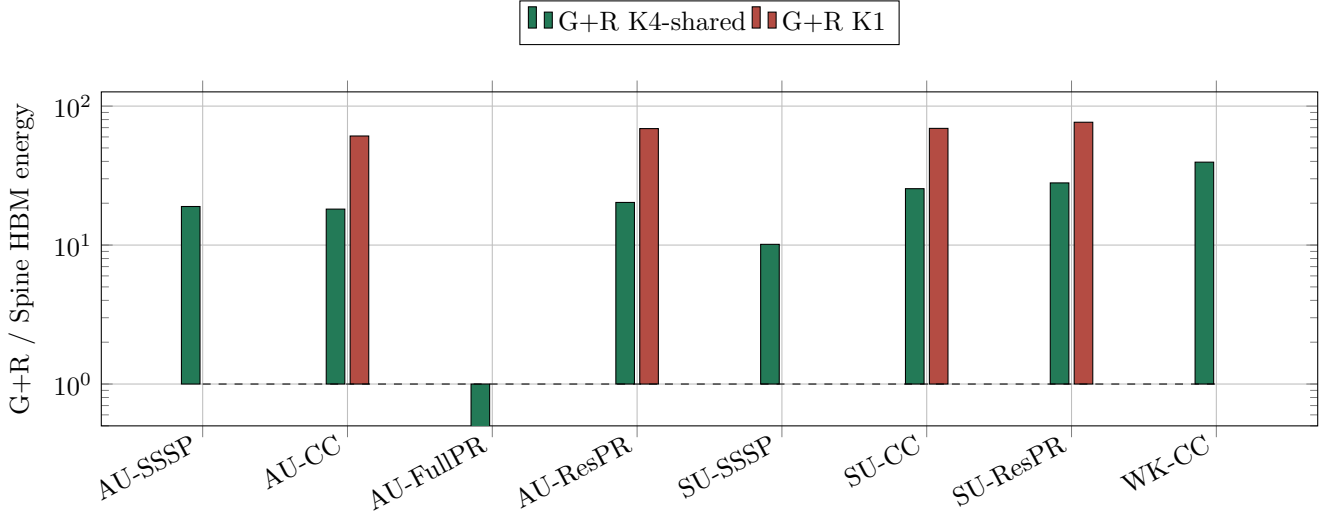


Figure 3: All-controller DRAMSim3 energy ratio. This is HBM energy, not total accelerator or board energy.

3 Update-only tradeoff

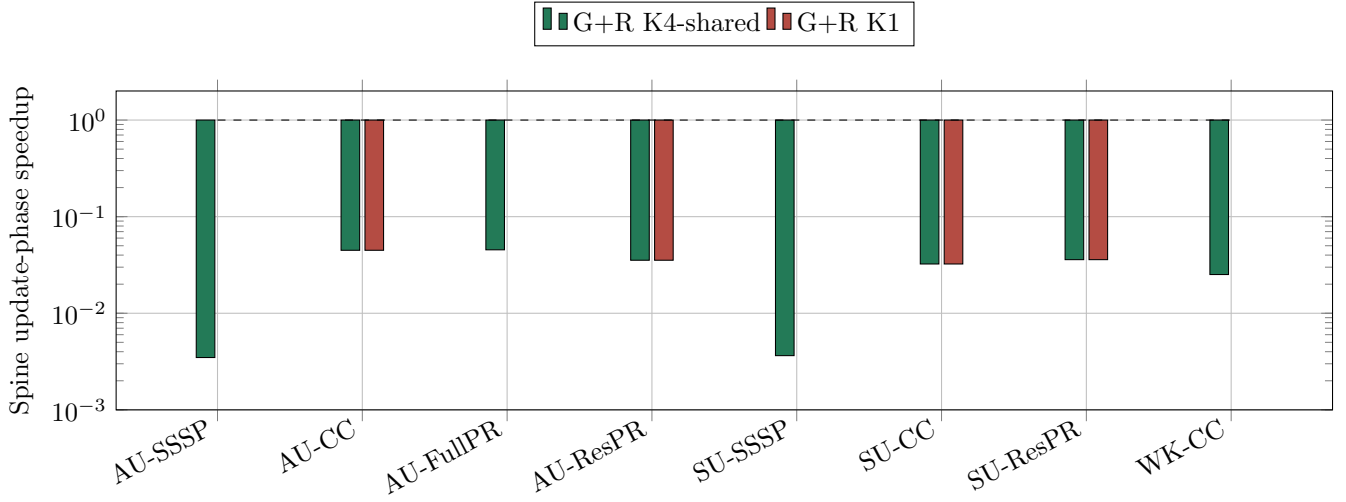


Figure 4: Update-phase cycles only. Values below one expose Spine’s current maintenance cost even when end-to-end execution favors Spine.

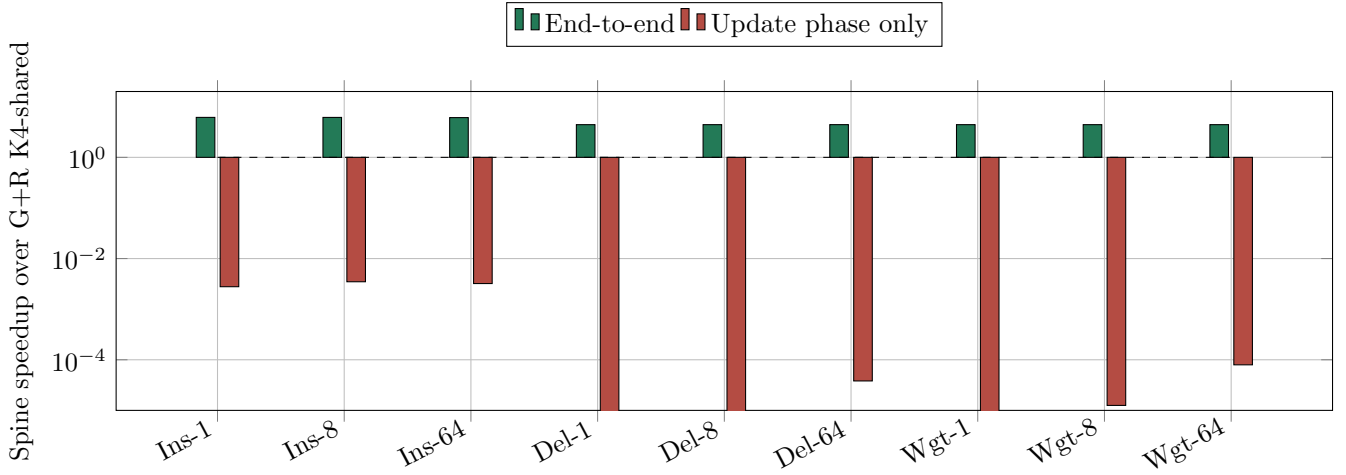


Figure 5: AskUbuntu weighted-SSSP operation and batch sweep. Ins, Del, and Wgt denote insertion, deletion, and weight change. Update speedup uses semantic user mutations and the isolated update-phase cycle window.

4 Dense-batch behavior

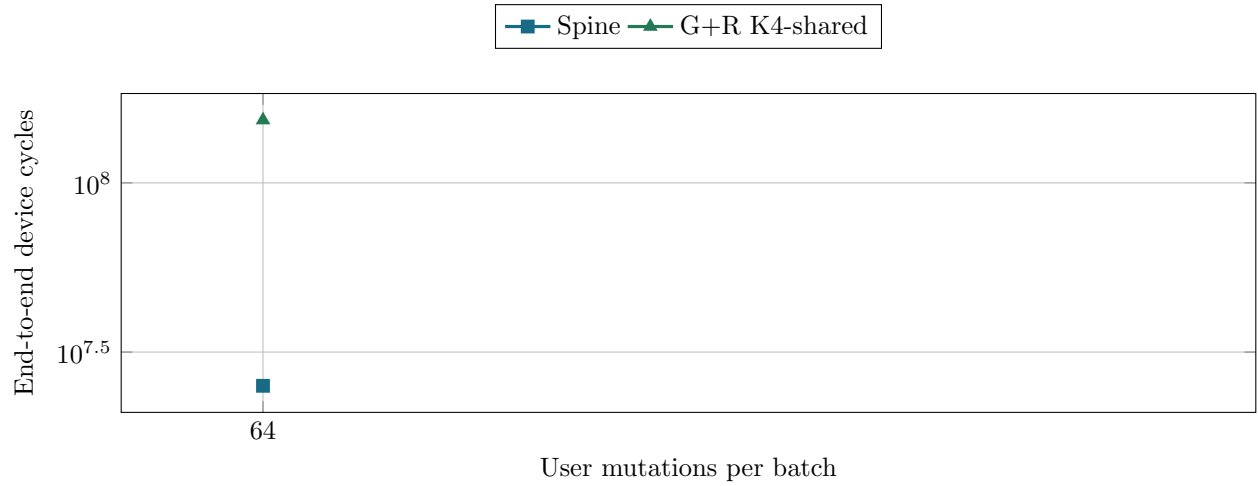


Figure 6: Correctness-gated AskUbuntu weighted-SSSP insertion latency as the batch grows from 64 toward 4096 mutations. Missing points are still running, not zero-valued measurements.

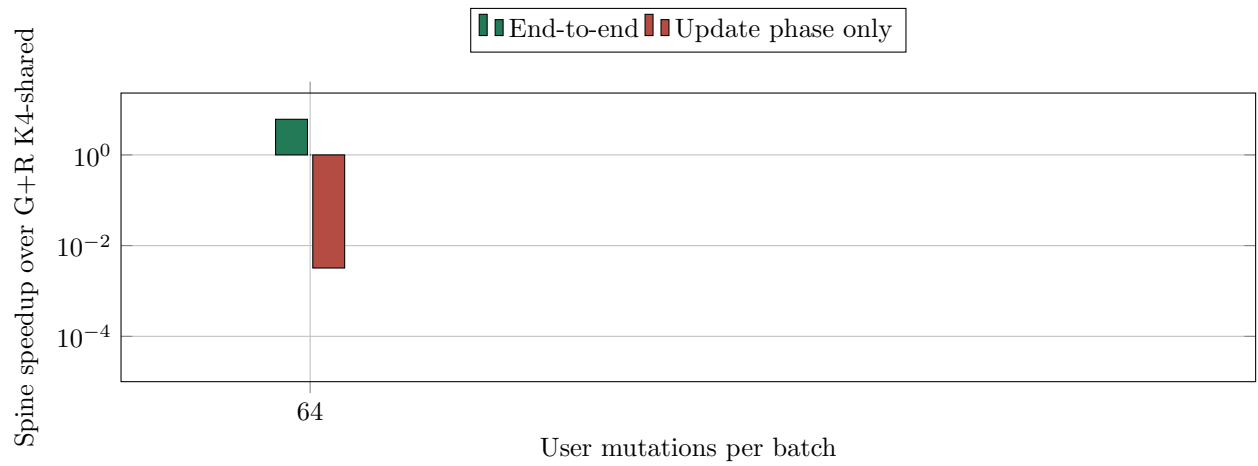


Figure 7: Dense-batch tradeoff. End-to-end and isolated update-phase windows are kept separate because the current Spine maintenance path can lose even when its complete execution wins.

5 Correctness coverage

Algorithm	Cross-system groups	Exact	K1/K4 triplets	Failed
SSSP	10	10	0	0
CC	3	3	2	0
FullPR	2	1	0	0
ResPR	2	2	2	0

Table 1: Only groups containing at least two systems are counted. Exact means canonical final-state equality after both architecture and independent mathematical-oracle admission.

6 Frozen dataset catalog

Data	Vertices	Directed edges	CC edges	ResPR edges	Spine full
AU	515281	390847	679246	826848	yes
SU	567316	616118	1061248	1075035	yes
WK	1140149	1142352	2032558	2190639	yes
SO	6024271	23724166	41008044	28299669	yes
BC	24575382	38968503	77937006	52778301	no
HW	1139905	112751422	112751422	112784084	yes
PK	1632803	44603928	44603928	44603928	yes
OK	3072441	234370166	234370166	234370166	yes
LJ	4847571	85702474	85702474	85703436	yes
LJ08	5363260	99028542	99028542	99028616	yes
UK	18520486	292243663	523574516	295209230	no
R19	524288	15483485	29732038	15672181	yes

Table 2: Immutable materialized graph views. Full PageRank uses the separately frozen 4M-edge cap. “Spine full” is the current 2^{24} -vertex admission check, not a performance result.

7 Simulator engineering runtime

System	Completed runs	Median host s	Maximum host s
Spine	13	174.4	5321.9
G+R K4-shared	10	1533.5	10388.1
G+R K1	5	2206.8	5867.6

Table 3: Observed host wall time under concurrent campaign load. These values measure simulator throughput and are not accelerator performance.

Claim boundary. Only complete correctness-gated pairs enter the figures. Current FPGA/RTL evidence supports feasibility and PPA separately; this report does not claim cycle-for-cycle FPGA calibration or ASIC total power.